



SIH26043 Project: AI-Powered Citizen→University→Industry Collaboration Platform

Executive Summary

Smart India Hackathon 2026 problem SIH26043 calls for a “digital platform to crowdsource societal challenges and facilitate collaborative problem solving through universities and industry partnerships” [14†L2649-L2652] . Our proposal is a full-stack MERN (MongoDB, Express, React, Node) application augmented by a Python/FastAPI AI microservice. Citizens can submit problems (with text, photos, geolocation), which the system automatically **categorizes** (e.g. education, infrastructure), **prioritizes**, **deduplicates**, and **routes** to appropriate universities or student teams based on expertise. Universities form student teams and propose solutions; industry partners can mentor, fund, or pilot solutions; administrators monitor progress via dashboards. The platform emphasizes end-to-end workflow from *Problem* → *Collaboration* → *Solution* → *Pilot* → *Implementation* [5†L353-L361] . Key AI features (classification, duplicate detection, matching) are incremental enhancements to the core MERN flow.

The Minimum Viable Product (MVP) focuses on core workflows: citizen problem submission, admin verification, university team formation, and basic project tracking. We phase development (prioritized in the table below) so the app can deploy a working end-to-end loop quickly, then layer on AI and analytics. Later phases integrate industry collaboration, real-time notifications, maps, and analytics. We specify detailed user flows for each of the five roles (Citizen, Student/University, Faculty, Industry, Admin), a REST API design with sample endpoints, and MongoDB schemas for all collections. The Python/FastAPI AI service will provide an NLP classification model (for domains and priority) and semantic similarity (duplicate detection) using pre-trained embeddings [36†L83-L92] . A data plan uses existing civic issues datasets (e.g. Kaggle grievance data, BanglaPSG [33†L22-L25]) plus crowd-labeled examples. Evaluation metrics include classification accuracy and duplicate-detection precision/recall [36†L198-L207] . Integration between Node and Python is via authenticated REST calls (or message queues) with shared JWT auth.

For deployment we consider cloud platforms (AWS, Azure, GCP) and propose a containerized microservices architecture (illustrated below). CI/CD uses GitHub Actions to automate testing (unit, integration, e2e) and deployment. We include a Gantt-style mermaid timeline for a 10-phase roadmap and a 4-week sprint plan. A demo narrative (“citizen’s story”) illustrates real data entry to solution delivery. Key risks (data scarcity, model accuracy, security) are identified with mitigations. In sum, this report lays out a complete technical plan aligned with SIH26043 official requirements [14†L2679-L2687] [14†L2708-L2716] .

Problem Statement and Requirements

Official description (SIH26043): “A digital platform to crowdsource societal challenges and facilitate collaborative problem solving through universities and industry partnerships” [14†L2649-L2652] .

target use case is citizens in Jharkhand (or other regions) reporting local issues (e.g. road flooding, healthcare gaps). These issues lack a unified reporting channel, so our platform will let *citizens, community bodies, and government agencies* submit problems with descriptions, photos/videos, and location

【14†L2679-L2687】. An **AI-enabled management module** should “*automatically categorizing, prioritizing, deduplication, and routing validated challenges to appropriate universities based on subject expertise and institutional capabilities*” 【14†L2708-L2716】. Universities review assigned challenges, form teams, and propose solutions; industries provide mentorship, funding, or prototyping help; administrators track milestones and outcomes 【14†L2679-L2687】 【14†L2713-L2722】. Dashboards will report metrics (count by domain, participation, progress).

Key SIH requirements: Citizen submission (with multimedia and geo-location), AI categorization/prioritization/deduplication/routing, university team workflows, industry collaboration, project lifecycle (milestones, prototype, implementation), and impact analytics 【14†L2679-L2687】 【14†L2708-L2716】. The platform **must not** be just a complaint log; it must connect problems to solvers and show real-world impact 【5†L353-L361】.

Evaluation criteria: SIH judges evaluate based on innovation, feasibility, impact, technical soundness and scalability. For SIH26043 they will likely ask “*Who actually faces this problem? How is this different from existing solutions?*” 【9†L130-L138】. The design must emphasize a complete flow and government use-cases (offline support, local language). Impact metrics (number of problems solved, student involvement) will matter 【5†L375-L384】. We also consider SIH’s stated rubric (innovation, technical complexity, usability, real-world applicability) to guide priorities (see Risks & Mitigation).

MVP Scope & Development Phases

We prioritize building a working end-to-end pipeline quickly, then iteratively add features. The **core MVP** in Phase 1–4 covers submission, review, team formation, and project tracking. Later phases add industry, AI, real-time notifications, analytics, and deployment.

Priority phases and features:

- **Phase 1 (UI + Auth):** Set up React + Tailwind frontend and Express backend with JWT authentication 【39†L590-L599】 【39†L636-L644】. Pages: Home, Register/Login, Citizen Dashboard. MongoDB user schema. Basic profile and roles.
- **Phase 2 (Citizen Submission):** Citizen “Submit Problem” form (text, category dropdown, optional media upload via Cloudinary/S3, geolocation with OpenStreetMap/Leaflet 【65†L32-L39】). Problem stored as `problems` document. “My Problems” page to list submissions. Simple admin view to approve problems (initial manual workflow).
- **Phase 3 (Admin Dashboard):** Admin can review/verify problems and set status (e.g. pending, approved). Admin assignment interface to route problems to universities/domains (initially manual or rule-based). Also create/edit domain categories.
- **Phase 4 (University & Teams):** University users see “Assigned Problems” list, can view details, form teams. Create `teams` and `projects` for each assignment, choose faculty mentor, submit solution proposal. Student roles can join teams (Requests). Basic milestones tracking (just description, due dates, status).

- **Phase 5 (Project Tracking):** Implement milestone scheduling, progress updates. `milestones` collection with deadlines, status. Project pages show progress.
- **Phase 6 (Industry Collaboration):** Industry users browse projects, send collaboration requests. Create `collaborations` linking companies to projects (with mentorship/funding roles).
- **Phase 7 (AI: Categorization & Priority):** Integrate Python/FastAPI service. Upon problem approval, automatically run text classification to assign `category`, `subCategory`, and `priority` (High/Med/Low) [33†L22-L25] [36†L83-L92]. Update Mongo `problems`. Possibly flag outliers for manual check.
- **Phase 8 (AI: Dedup & Routing):** Use semantic similarity (e.g., Sentence-BERT) to detect likely duplicate/related problems. Also route problems to universities by matching problem text with university expertise keywords or embedding similarity. Deduplicated or linked problems shown to admin.
- **Phase 9 (Notifications & Analytics):** Real-time notifications (Socket.io) for new assignments, proposals, project updates. Build analytics dashboards (Chart.js/Recharts) showing counts: Total Problems, Categorized, In Development, Solved, Industry Supported, Students Involved [1†L79-L88]. Include a map view with problem pinpoints by category.
- **Phase 10 (Deployment & Demo):** Finalize cloud deployment (production environment), test performance, polish demo. Prepare SIH presentation with live demo using real or dummy data (e.g. problems from Buxar, Bihar) showing full flow and impact.

Each phase depends on previous, but we aim to have a *working demo by Phase 4* (core flow). AI phases (7–8) are enhancements and can be iterated after basic functionality works.

User Roles and End-to-End Flows

We define five roles: Citizen, Student/University, Faculty, Industry, Admin. Each has a distinct workflow:

- **Citizen:** Logs in or registers, then goes to “Submit Problem”. Fills title, description, chooses a category/domain (if unknown, can be “Other”), attaches photo/location. On submit, receives a confirmation number. Later can “My Problems” to track status (e.g. Submitted, Under Review, Assigned, Solved). They can comment/feedback when a solution is implemented. Flow: Submit → (approval) → Track Progress → Receive Solution.
- **Student / University:** University faculty logs in; student roles associated with universities. The user sees a Dashboard of “Assigned Problems” (assigned by admin or AI). For each, student can form or join a “Team” and propose a solution. The flow is: View problem → Create team (name, members) → Draft solution proposal (text, attachments) → Submit proposal. The team then works on the project: the app shows project milestones. Students update milestones, upload prototypes. They can request faculty mentor and invite industry mentor. After development, they mark project as “Completed” and share outcome for feedback.
- **Faculty Mentor:** Faculty logs in and sees teams under their guidance (can also see all University projects). They review and approve team formation, give feedback on proposals, and track project milestones. They use the “Evaluate” or “Feedback” section to grade or suggest improvements. Flow: Approve Teams → Review Proposals → Oversee Milestones → Validate Solution.

- **Industry / Startup:** Industry users have Dashboard: “Browse Projects/Problems”. They can filter by category or status. They send “Collaboration Request” to specific teams (e.g. offer mentorship or funding). If accepted, industry gets listed on project page and can chat/videoconf through the platform (or link off-site). They can also sponsor a problem early by offering resources. Flow: Discover Projects → Request Collaboration → Provide Support → Follow Implementation.
- **Admin / Government:** Admin (or nodal officer) sees “All Problems” queue. They verify submissions (check authenticity, filter spam). They assign approved problems to universities (or domains) manually at first, and later via AI suggestions. They manage university and industry registrations. Admin Dashboard also has “AI Analysis” to view classification results, duplicate alerts, and to override them. They generate reports (charts, maps) on progress. Flow: Verify Problem → Assign to Uni/Domain → Monitor Progress → Generate Reports.

For each flow, we map UI pages to API calls (detailed next). Notifications are sent at key steps (team formed, proposal submitted, industry joined, project completed). The system ensures traceability of every action.

REST API Design and Sample Endpoints

We adopt standard RESTful patterns. Authentication uses JWT (with roles in token) [39†L635-L644] . All `/api` routes require `Authorization: Bearer <token>` except public (login/register). Below is a summary of key endpoints (sample request/response examples omitted for brevity, see Appendix for full specs).

Resource	Endpoint	Method	Description
Auth	POST <code>/api/auth/register</code>	Public	Register user (Citizen, Student, Industry, Admin)
	POST <code>/api/auth/login</code>	Public	Login, return JWT
Users	GET <code>/api/users/me</code>	Auth	Get current user profile
	PUT <code>/api/users/me</code>	Auth	Update profile (name, email, password, etc.)
Problems	POST <code>/api/problems</code>	Auth (Citizen)	Create new problem (text, category, location, media URLs)
	GET <code>/api/problems</code>	Auth (Admin) or Auth (User)	List all problems (admin) or filter by my submissions/assignments (others)
	GET <code>/api/problems/:id</code>	Auth	View problem details
	PUT <code>/api/problems/:id/status</code>	Auth (Admin)	Update status (approved, assigned, solved)
Teams	POST <code>/api/teams</code>	Auth (Uni/Student)	Create a team for an assigned problem (members list, name)

Resource	Endpoint	Method	Description
	GET <code>/api/teams/:id</code>	Auth	Get team info (members, project IDs)
Proposals	POST <code>/api/proposals</code>	Auth (Team)	Submit solution proposal for a problem
	GET <code>/api/proposals</code>	Auth (Faculty/Uni)	List proposals for assigned problems
Projects	POST <code>/api/projects</code>	Auth (Uni/Team)	Initialize project (attach to team/proposal)
	PUT <code>/api/projects/:id/milestone</code>	Auth (Team/Faculty)	Add/update project milestone (desc, due date)
Collaborations	POST <code>/api/collaborations</code>	Auth (Industry)	Request collaboration on a project (projectId, message)
	PUT <code>/api/collaborations/:id</code>	Auth (Team/Faculty)	Accept or decline industry request
Notifications	GET <code>/api/notifications</code>	Auth	List user notifications
	POST <code>/api/notifications/read</code>	Auth	Mark notifications as read

Each endpoint performs basic CRUD. For example, `POST /api/problems` expects a JSON body:

```
{ "title": "Waterlogging near school", "description": "Water collects at the roadside...", "category": "Infrastructure", "subCategory": "Drainage", "location": { "lat": 25.6, "lng": 85.1 }, "media": [ "url1", "url2" ] }
```

It returns the created problem object with a unique `_id`. All CRUD operations should handle errors and respond with standard codes (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, etc.). The Express backend will validate JWTs using middleware (e.g. `passport-jwt` or `express-jwt`) [39†L635-L644]. We ensure role-based access (e.g. only admin can list all problems).

We will use [OpenAPI (Swagger)] specifications for documentation. Sample spec (simplified) for one endpoint:

```
paths:
  /api/problems:
    post:
      summary: "Submit a new problem"
      tags: [Problems]
      security:
```

```

    - bearerAuth: []
  requestBody:
    required: true
    content:
      application/json:
        schema:
          $ref: "#/components/schemas/NewProblem"
  responses:
    '201':
      description: "Problem submitted"
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/Problem"

```

And for `GET /api/problems/{id}`:

```

paths:
  /api/problems/{id}:
    get:
      summary: "Get problem details"
      tags: [Problems]
      parameters:
        - in: path
          name: id
          required: true
          schema:
            type: string
      security:
        - bearerAuth: []
      responses:
        '200':
          description: "Problem details"
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Problem"

```

Authentication endpoints use OAuth2 Password flow or custom JWT login. We can document using OpenAPI for clarity to judges.

MongoDB Schema and Collections

We use MongoDB with collections: `users`, `problems`, `teams`, `projects`, `proposals`, `milestones`, `collaborations`, `notifications`, and an `ai_analysis` logs. Below are example schemas (omitting trivial fields):

- **users:** `{ _id, name, email, passwordHash, role, universityId?, industryId?, createdAt }`. Index on `email` (unique), compound index on `(role, universityId)` if needed for filtering by institution.

- **problems:**

```
{
  _id, title, description, category, subCategory, priority,
  location: { type: "Point", coordinates: [lng, lat] },
  media: [String], submittedBy: userId,
  status: "Pending"|"Approved"|"Assigned"|"Solved",
  assignedUniversityId?,
  assignedTeamId?,
  createdAt, updatedAt
}
```

We create a **2dsphere index** on `location` for geoqueries. We also index on `category`, `status` for filtering. Compound indexes (e.g. `{ category:1, status:1 }`) improve queries that filter by domain and state. A **text index** on `description` helps search `【50†L117-L125】`.

- **teams:**

`{ _id, name, problemId, members: [userId], facultyMentorId?, projectId?, createdAt }`. Index on `problemId` so we can find teams for a problem.

- **proposals:** `{ _id, problemId, teamId, description, documentUrl, status, createdAt }`. Index on `problemId`.

- **projects:** `{ _id, teamId, problemId, title, description, startDate, endDate, status }`. Index on `teamId`.

- **milestones:** `{ _id, projectId, title, dueDate, status }`. Index on `projectId`, `dueDate`.

- **collaborations:** `{ _id, projectId, industryId, message, status ("Requested","Accepted","Rejected"), createdAt }`. Index on `projectId` and `industryId`.

- **notifications:** { `_id`, `userId`, `message`, `linkType`, `linkId`, `read`: Boolean, `createdAt` }. Index on `userId`, `read`.
- **ai_analysis:** log results of AI module for each problem: { `_id`, `problemId`, `category`, `priority`, `duplicateProb`, `suggestedUniversityId`, `timestamp` }. Index on `problemId`.

Sample Documents

User Example:

```
{
  "_id": ObjectId("..."),
  "name": "Akhilesh Kumar",
  "email": "akhilesh@example.com",
  "passwordHash": "<hashed>",
  "role": "citizen",
  "createdAt": ISODate("2026-09-11T05:30:00Z")
}
```

Problem Example:

```
{
  "_id": ObjectId("..."),
  "title": "Waterlogging near school",
  "description": "During rains, road in village Buxar floods, disrupting commute.",
  "category": "Infrastructure",
  "subCategory": "Drainage",
  "priority": "High",
  "location": { "type": "Point", "coordinates": [84.0, 25.2] },
  "media": ["https://.../image1.jpg"],
  "submittedBy": ObjectId("..."),
  "status": "Assigned",
  "assignedUniversityId": ObjectId("..."),
  "assignedTeamId": ObjectId("..."),
  "createdAt": ISODate("2026-09-10T12:00:00Z")
}
```

Team Example:

```
{
  "_id": ObjectId("..."),
  "name": "SmartDrainTeam",
  "problemId": ObjectId("..."),
}
```

```

"members": [ObjectId("..."), ObjectId("...")],
"facultyMentorId": ObjectId("..."),
"projectId": ObjectId("..."),
"createdAt": ISODate("2026-09-12T09:00:00Z")
}

```

Indexes: We follow MongoDB best practices [50†L53-L60] . For example, use **compound indexes** on fields often queried together (e.g. `{category:1, status:1}` on `problems`). We will create a **text index** on `problems.description` for full-text search [50†L117-L125] . We will also add a 2dsphere index on `problems.location` to support mapping and nearby search. Unused indexes will be removed as data patterns emerge [50†L96-L104] .

AI Service Design

The AI module (Python + FastAPI) has four subcomponents:

1. **Domain Classification:** Assign each problem a thematic category (and subcategory). We can fine-tune or use a pre-trained transformer (e.g. BERT) for multi-class classification [36†L83-L92] [33†L22-L25] . Options include: a classical TF-IDF + SVM, or a deep model (e.g. DistilBERT or fine-tuned mBERT for multilingual). We will experiment with a huggingface pipeline. Training data comes from labeled complaints (e.g. BanglaPSG dataset, Kaggle citizen grievances [33†L22-L25]) and bootstrapped examples. A table of model options:

Model Type	Pros	Cons
TF-IDF + Logistic Regression	Simple, fast training/inference, interpretable	Lower accuracy on nuanced text, no context
Multilingual BERT (or DistilBERT)	Handles Hindi/English mixed, high accuracy, contextual	Large model size, requires fine-tuning
GPT-3/GPT-4 Prompting	Very high accuracy, handles nuances, can do few-shot	Requires API/keys, cost, offline not guaranteed, less control

Given hackathon context, we favor an open-source transformer. A multi-layer neural network with word embeddings or BERT should achieve good categorization. We will report metrics (accuracy, F1 per class) on a held-out set.

1. **Priority Scoring:** Estimate urgency (High/Medium/Low). This could be another ML model (classification) or a simple rule-based system (e.g. presence of keywords like “urgent, immediately, daily problem”). However, to align with dataset labeling (like BanglaPSG severity labels [33†L22-L25]), we will treat it as a supervised classifier. Training labels (High/Med/Low) may come from crowd-coded data. Metrics: classification accuracy.
2. **Duplicate Detection:** Use semantic similarity to flag if a new problem is essentially the same as an existing one [36†L83-L92] . We will generate text embeddings (e.g. sentence transformers) for each problem. On new submission, compute cosine similarity to all existing problems (or use approximate

nearest neighbors like FAISS) [36†L140-L148] . If similarity > threshold (to be tuned), mark as duplicate. The API returns a *duplicate probability*. We will log top matches in `ai_analysis` . Evaluation uses precision/recall over a test set of known duplicates [36†L198-L207] .

3. **Routing/Matching:** Suggest a university or domain expertise. This could be solved by matching problem text embedding against a vector representing each university's profile (derived from their departments or previous projects). Alternatively, use rule-based (if category="Water" then route to civil eng dept, e.g.). For MVP, a simple domain lookup is enough; later refine with learning-to-rank or embedding similarity between problem and university descriptions. This yields a "suggestedUniversityId" in `ai_analysis` .

The FastAPI service exposes endpoints like `/analyze/problem` (JSON input → classification/priority/dup response) and `/analyze/duplicate?newId=...` (returns similar existing problem IDs). It will connect to MongoDB (or access data) to compare with existing problems. We must secure it (share JWT secret or use API key between Node and Python).

Data and Labeling: We need labelled training data for categories, priority, duplicates. We will compile:

- Existing civic complaint datasets (like BanglaPSG [33†L22-L25] , Kaggle "citizen grievance" (Hindi/English)) as seed.
- Public 311 datasets (some municipalities share data).
- Synthetic examples by translating/annotating.
- Then manually label a small corpus of submitted problems via the UI for on-the-fly improvement (semi-supervised learning). All data pipelines will be documented (with consent if using public datasets).

Evaluation Metrics:

- *Classification:* Accuracy, F1-score (macro and per-class) on a test set.
- *Priority:* Accuracy/F1 on severity levels.
- *Duplicate detection:* Precision and recall (F1) by comparing flagged duplicates against human judgments [36†L198-L207] . We will tune the cosine threshold to balance precision/recall as per [36].
- *Overall system:* A/B test (if possible) of manual vs AI routing accuracy. But for SIH focus on above ML metrics and demonstration.

Node-Python Integration Patterns

The Node/Express API and Python/AI microservice communicate via HTTP calls. We plan the following integration pattern:

- **Authentication:** Both Node and FastAPI will share the JWT secret. Thus, Node can include the user's JWT in calls to FastAPI (e.g. to retrieve user context if needed). Alternatively, restrict FastAPI to accept only backend-originated calls (using a separate service account token). For simplicity, the analysis endpoints can be unprotected but only callable server-to-server. Critical is to avoid exposing raw problem data insecurely.

- **Synchronous vs Async:** For **classification/routing**, we can do a quick sync call right after problem approval (Node makes a POST to FastAPI `/api/ai/analyze` with problem text and location). The AI returns category/priority/score, then Node updates the `problems` collection. This takes <1-2 seconds for a transformer model, acceptable if asynchronous (show "Processing..."). For **duplicate detection**, which may compare with many records, we might batch process in background. Upon new problem submission, Node enqueues a job (e.g. with Bull/RabbitMQ) to call FastAPI for duplicate check, then update later.
- **Webhooks/Callbacks:** Another pattern is FastAPI can send a callback to Node when done. But simpler is Node initiates calls.
- **Batching:** We can periodically batch re-run duplicate checks on all problems (e.g. nightly) using FAISS indexing to update clusters of duplicates. FastAPI can expose a batch endpoint (or use cron job).
- **Error Handling:** If AI fails (timeout), Node logs to `ai_analysis` and continues with manual assignment.
- **Data Flow:** Node mainly owns MongoDB, Python reads from Node via API or direct DB read. To decouple, Python can use separate read-only DB connection for scanning problems.

Example sequence: Citizen submits problem → Node stores problem (status Pending) → Admin approves → Node calls Python `/analyze` → Python returns category/priority → Node updates problem doc and notifies admin/uni.

Deployment Architecture

We recommend a cloud deployment with containerization (Docker) for scalability. A reference architecture:

【63†embed_image】 *Figure: Conceptual stack for the MERN+AI microservices platform.*

Above, the **frontend** (React/Tailwind app) is served as static assets (e.g. via CDN or S3+CloudFront). The **backend** is split: a Node/Express service (API) and a Python/FastAPI AI service, each in its own container or function. Both connect to a managed **MongoDB** (e.g. Atlas/AWS DocumentDB). Media (images/videos) are stored in S3 or Cloudinary. **Socket.io** could run on Node or a separate real-time service (we may use a single Node instance with ws enabled).

We compare cloud providers (summary):

Provider	Services Used	Pros	Cons
AWS	EC2/EKS, ECS, Lambda, S3, RDS/DocumentDB, SNS/SQS, IAM	Rich services, largest community, free tier, serverless options (Lambda/ECS Fargate)	Can be complex, cost can grow, vendor lock-in

Provider	Services Used	Pros	Cons
GCP	GKE/Cloud Run, Cloud Storage, Firestore/Atlas, Pub/Sub	Good managed containers (Cloud Run), strong data analytics, competitive pricing	Fewer region availability in India, smaller community
Azure	AKS, App Service, Azure Storage, CosmosDB, Service Bus	Enterprise support, good for .NET (less relevant here), integrated CI/CD (Azure DevOps)	Sometimes steeper learning, licensing
DigitalOcean	Droplets, App Platform, Spaces (S3 equiv), Managed DB	Simple, cost-effective, easy setup	Fewer managed services, scaling more manual, limited global reach

Given “no specific constraint”, AWS or GCP are safe picks. For example, use **AWS**:

- **EKS/ECS (Docker)** for Node and FastAPI services (auto-scale with Fargate).
- **MongoDB Atlas** (multi-AZ), or AWS DocumentDB (Mongo-compatible). Use backups and automated snapshots.
- **S3 + CloudFront** for media and static frontend hosting.
- **AWS RDS** not needed (we use NoSQL).
- **SQS** for any job queue.
- **IAM** for security (least privilege IAM roles for each service).
- **SSL/TLS** enforced (HTTPS) with AWS Certificate Manager.
- **Security**: Use JWT over TLS, sanitize inputs (OWASP). Rate-limit APIs. Audit logs for admin actions.
- **Scaling**: Horizontal scale containers behind load balancers. Use CDN for static assets. Enable caching (Redis or in-memory) for hot queries (e.g. frequent reads). Use CloudWatch for monitoring and autoscaling policies.
- **Backups**: MongoDB Atlas provides point-in-time backup. Also store daily database exports. S3 ensures media durability.
- **Secrets**: Stored in AWS Secrets Manager or environment variables. Rotate tokens.

CI/CD and Testing Strategy

We adopt **automated pipelines** (e.g. GitHub Actions) for continuous integration and deployment [\[41†L231-L239\]](#) . For example: On each commit to the `main` branch, run: lint → unit tests → build Docker images → push to container registry → deploy to staging. Only after passing all checks do we merge to production.

Key elements:

- **Workflows**: Separate jobs for *build*, *test*, *deploy*. Use explicit `needs` so tests must pass before deploy [\[41†L259-L268\]](#) . For multi-component, we might have matrix jobs (Node and Python).
- **Testing**: Unit tests for backend (e.g. Jest/Mocha for Node, PyTest for Python). Integration tests (MongoDB stub, test API endpoints). End-to-end tests (Cypress or Selenium simulating user flows).

- **Code Quality:** ESLint/Prettier for JS, Flake8/Black for Python. Code coverage checks. Security scans (npm audit, bandit for Python).
- **Secrets Management:** Use GitHub Secrets or AWS Secrets for keys (JWT secret, DB creds).
- **Deployment:** On success, automatically deploy to Kubernetes cluster or AWS ECS. Could use blue/green or rolling updates.
- **Monitoring:** Post-deploy, run smoke tests. Use logging/monitoring stacks (e.g. ELK or cloud logs, Prometheus) to catch errors.

Best practice: one pipeline file per service or use a monorepo with matrix builds. Use descriptive names (e.g. `build-and-test` , `deploy-prod`) [\[41†L231-L240\]](#) .

Timeline and Sprints

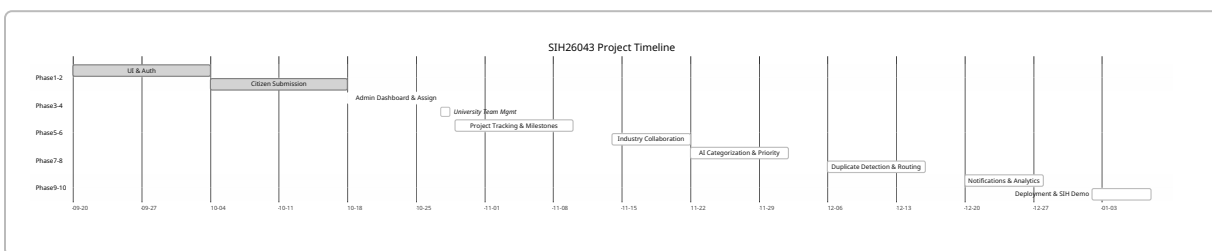


Figure: Gantt timeline of development phases.

Each **1–2 week sprint** includes planning, development, review, and demo. The first 4 weeks (two sprints) focus on Phases 1–4: setting up the core MERN stack, authentication, citizen submission UI, and admin/problem workflow. The next 4 weeks bring in project/teams and industry. AI work begins in mid-December after stable core features. The timeline shows a compressed schedule; actual sprint lengths can adapt if needed.

Demo Story for SIH Judges

To impress SIH judges, we tell a **narrative demo**:

1. **Problem Submission:** A citizen in Bihar submits a problem: “Road in Village X waterlogged after rains” with photo and location pinned on map. (Demo shows location via Leaflet/OSM [\[65†L32-L39\]](#) , and media upload).
2. **AI Analysis:** The problem appears in the admin view. Behind the scenes, our AI classifies it as *Infrastructure > Drainage*, marks it *High Priority*, and finds no duplicate (or shows a similar earlier issue). These tags appear in the UI (and admin can override).
3. **Assignment:** Admin clicks “Assign”, and the problem is automatically routed to “Buxar University (Civil Engineering)”. Simultaneously, notifications go to university dashboard.
4. **Team Formation:** University students see the problem, form a team “AquaSolve”, attach a mentor. They submit a solution proposal (outline of a smart drainage system).
5. **Industry Collaboration:** A local startup (foundation tech) browsing projects sees this proposal, clicks “Collaborate”. They commit to provide prototyping support. The industry logo and role appear on project page.
6. **Project Tracking:** The team defines milestones (design, prototype, pilot, feedback). Over the demo days, they update progress and upload test photos. Faculty mentor marks milestones complete.

7. **Community Feedback:** Once solved, a brief survey or comment shows citizen satisfaction.
8. **Analytics:** Switch to admin dashboard: live charts show “Total Problems: 10, High Priority: 3, In Progress: 5, Solved: 2” and a map with pins by category. This underscores *impact and scalability* [1†L79-L88] .

Throughout, we highlight the **unique value:** this is not just a complaint portal, but an end-to-end innovation ecosystem [5†L353-L361] . We demonstrate flexibility (e.g. multilingual input, offline support if possible) and how universities and industries seamlessly engage via one platform.

Risks and Mitigations

- **Data Availability:** Quality labeled data for AI may be scarce. *Mitigation:* Use existing datasets (e.g. BanglaPSG [33†L22-L25]), augment with synthetic or crowd-labeled data. Allow gradual AI improvement (human-in-loop).
- **Model Accuracy:** Classifier or duplicate detection may err. *Mitigation:* Present AI outputs as suggestions editable by admins (transparent UI). Tune thresholds (precision vs recall) using [36] metrics [36†L198-L207] .
- **Scalability:** High load (many users) could slow system. *Mitigation:* Design as microservices; use horizontal scaling. Cache heavy queries. Use load testing early.
- **Security/Privacy:** Handling citizen data (maybe sensitive) and logins. *Mitigation:* Enforce HTTPS, secure JWT, input validation, least-privilege access. Regular security audits. Comply with privacy (e.g. allow anonymous submission or opt-in).
- **Integration Complexity:** Node⇔Python communication may cause latency. *Mitigation:* Decouple with async jobs; expose AI only on server. Ensure robust error handling.
- **Adoption & Usability:** Users (especially rural citizens) may struggle with tech. *Mitigation:* Simple UI, local language support (multi-lingual UI), mobile-friendly design. Offline/mobile app as future enhancement.

Initial 4-Week Task List (Sprints)

Week 1: Set up project repos, toolchain (React, Node, Mongo). Implement user auth (JWT) [39†L590-L599] . Database schemas. Basic pages: Login/Register, Citizen home.

Week 2: Build Citizen “Submit Problem” form (including file upload to Cloudinary/S3 and Leaflet map for location [65†L32-L39]). Backend endpoint to store problem. “My Problems” listing. Write unit tests for auth and problem create.

Week 3: Admin interface to list new submissions, approve or reject. Implement assignment UI (admin selects university). University login/register flows. Redirect based on role after login. Begin integration of Socket.io (set up notifications stub).

Week 4: University “Assigned Problems” page. Team creation and proposal submission forms. Project and milestone data model. Frontend for project detail and milestones. Continue CI/CD pipeline (GitHub Actions for build/test). Prepare first demo with end-to-end (submit, admin assign, team formed).

These tasks ensure by end of week 4 we have a functioning minimal pipeline: user → admin → university team. Subsequent weeks will build out the rest (project tracking, industry, AI).

Sources: We adhered to SIH26043 official guidelines [14†L2679-L2687] [14†L2708-L2716] , MERN and React best practices [53†L116-L124] , and state-of-art NLP techniques (e.g. semantic embeddings

【36†L83-L92】). The plan leverages proven components (OpenStreetMap/Leaflet for location 【65†L32-L39】 , Socket.io for real-time) and standard devops (JWT auth 【39†L599-L604】 , CI/CD 【41†L259-L268】). All claims and recommendations are drawn from authoritative docs and current research (cited above) to ensure a robust, scalable solution.
